國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 06

姓名：紀宥丞

學號：114310221

Lab06-Ex1.
Blackjack (Use Queue)
Code

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <random>
#include <ctime>
using namespace std;

struct Player {// 玩家結構體，包含名稱和分數
	string name; // 玩家名稱
	int score;	// 玩家分數
};

class Card {//建立一個 Card 類別，用來表示單張撲克牌
public:
	string suit; // 花色
	string rank; // 點數
	Card() {}
	Card(string s, string r) : suit(s), rank(r) {}
	void display() const {
		cout << rank << " of " << suit << endl;
	}
};

class Deck {// 牌組類別，負責生成和洗牌
private:
	Card cards[52]; // 52 張牌
	int index; // 目前發牌索引
public:
	Deck() : index(0) {// 初始化牌組
		string suits[] = { "Hearts", "Diamonds", "Clubs", "Spades" };// 四種花色
		string ranks[] = { "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A" };// 13
張牌
		int k = 0;
```

```
        for (int i = 0; i < 4; i++) { // 4 種花色
            for (int j = 0; j < 13; j++) { // 13 種點數
                cards[k] = Card(suits[i], ranks[j]);
                k++;
            }
        }
    }

    void shuffleDeck() {// 洗牌
        srand(time(0));
        for (int i = 51; i > 0; i--) {// 從最後一張牌開始，向前遍歷
            int j = rand() % (i + 1);// 生成隨機索引 j
            swap(cards[i], cards[j]);// 交換牌組中的兩張牌
        }
    }

    // 取得指定位置的牌
    Card getCard(int i) const {
        return cards[i];
    }
};

// Queue 類別
class Queue {
private:
    vector<Card> deck; // 用來存放卡牌的容器
    int Front;          // 指向牌堆的前端
    int Rear;           // 指向牌堆的尾端
    int capacity;       // 容器最大容量
public:
    // 建構函式
    Queue(int size) : Front(-1), Rear(-1), capacity(size) {}

    // 插入元素到佇列尾端
    void enqueue(Card card) {
        if (isFull()) {// 檢查是否滿
            cout << "牌堆已滿，無法插入！" << endl;
            return;
```

```
        }
        // 如果佇列是空的，初始化 Front 和 Rear
        if (Front == -1 && Rear == -1) {
            Front = 0;
            Rear = 0;
        } else {
            Rear++; // 尾端指標移動
        }
        deck.push_back(card); // 插入元素到尾端
    }

    // 從佇列中移除最前端元素
    void dequeue() {
        if (isEmpty()) { // 檢查是否空
            cout << "牌堆為空，無法移除！" << endl;
            return;
        }
        Front++; // 移除最前端元素只需要將前端指標向後移即可
    }

    // 獲取佇列最前端元素
    Card front() {
        if (isEmpty()) { // 檢查是否空
            cout << "牌堆為空，無法獲取最前端的牌！" << endl;
            return Card("", "");
        }
        return deck[Front];
    }

    // 判斷佇列是否為空
    bool isEmpty() const {
        return (Front == -1 && Rear == -1) || (Front > Rear);
    }

    // 判斷佇列是否已滿
    bool isFull() const {
        return deck.size() == capacity;
    }
```

```
};

void initializeDeck(Queue& cardDeck) {// 初始化並洗牌
	Deck deck;

	// 印出原始牌堆
	cout << "=== 初始牌堆 ===" << endl;
	for (int i = 0; i < 52; i++) {
		deck.getCard(i).display();
	}

	deck.shuffleDeck();// 洗牌

	// 印出洗牌後的牌堆
	cout << "\n=== 洗牌後的牌堆 ===" << endl;
	for (int i = 0; i < 52; i++) {
		deck.getCard(i).display();
	}
	cout << "==================\n\n";

	// 將洗好的牌放入 Queue 牌堆中
	for (int i = 0; i < 52; i++) {
		cardDeck.enqueue(deck.getCard(i));
	}
}

void initializePlayer(Player* player, string name, Queue& cardDeck) {// 初始化玩家，發兩張牌
	player->name = name;// 玩家名稱
	player->score = 0;// 玩家分數
	cout << player->name << " 抽到的牌: ";

	for (int i = 0; i < 2; i++) {// 發兩張牌
		// 從牌堆中取出一張牌，然後從牌堆中移除這張牌
		Card drawnCard = cardDeck.front();
		cardDeck.dequeue();

		// 將抽到的牌印出
```

```
        cout << "[" << drawnCard.rank << " of " << drawnCard.suit << "]    ";

        if (drawnCard.rank == "A") player->score += 1;//計算 A 分數
        else if (drawnCard.rank == "J" || drawnCard.rank == "Q" || drawnCard.rank == "K") player->score += 10;//計算 JQK 分數
        else player->score += stoi(drawnCard.rank);//計算 2-10 分數
    }
    cout << "\n";
    cout << player->name << "  玩家的初始分數: " << player->score << endl << endl;//印出玩家的初始分數
}

void playerTurn(Player* player, Queue& cardDeck) {//  玩家回合
    char choice;
    while (player->score < 21 && !cardDeck.isEmpty()) {//  玩家小於 21 點且牌堆不為空
        cout << player->name << "  您的手牌分數目前為: " << player->score << "，要抽牌嗎？(h =  抽, s =  停) : ";//顯示選擇
        cin >> choice;//輸入選擇

        if (choice == 'h') {//如果玩家選擇抽牌
            Card newCard = cardDeck.front();
            cardDeck.dequeue(); //  取出並移除

            if (newCard.rank == "A") player->score += 1;//計算 A 點數
            else if (newCard.rank == "J" || newCard.rank == "Q" || newCard.rank == "K") player->score += 10; //計算 JQK 點數
            else player->score += stoi(newCard.rank);//計算 2-10 點數

            //印出抽到的牌和總點數
            cout << player->name << "  抽到: [" << newCard.rank << " of " << newCard.suit << "]  總分: " << player->score << endl;
            if (player->score > 21) {//如果玩家大於 21 點
                cout << player->name << "  爆掉了！遊戲結束。\n";//玩家爆牌，莊家獲勝
                return;
            }
        }
        else if (choice == 's') {//如果玩家選擇停牌
```

```
                cout << player->name << "  選擇停牌，總分: " << player->score << endl;//印出總點數
                break;
            }
            else {
                cout << "請輸入有效選項 (h = 抽, s = 停)！" << endl;//輸入無效
            }
        }
}

void dealerTurn(Player* dealer, Queue& cardDeck) {// 莊家回合
    while (dealer->score < 17 && !cardDeck.isEmpty()) {// 莊家小於 17 點且牌堆不為空
        Card newCard = cardDeck.front();
        cardDeck.dequeue(); // 取出並移除

        if (newCard.rank == "A") dealer->score += 1;//計算 A 點數
        else if (newCard.rank == "J" || newCard.rank == "Q" || newCard.rank == "K") dealer->score += 10;//計算 JQK 點數
        else dealer->score += stoi(newCard.rank);//計算 2-10 點數

        //印出抽到的牌和總點數
        cout << "莊家抽到: [" << newCard.rank << " of " << newCard.suit << "] 莊家目前總分: " << dealer->score << endl;

        if (dealer->score > 21) {//如果莊家大於 21 點
            cout << "莊家爆了！玩家獲勝！" << endl;//玩家獲勝
            return;
        }
    }
}

// 判斷勝負
void determineWinner(Player* player, Player* dealer) {
    if (player->score > 21) cout << player->name << "  爆了！莊家獲勝！\n";
    else if (dealer->score > 21 || player->score > dealer->score) cout << player->name << " 贏了！\n";
    else if (player->score == dealer->score) cout << "雙方平手！\n";
    else cout << "莊家贏了！\n";
```

```
}

int main() {
    srand(time(0));
    Queue cardDeck(52);//宣告牌堆

    // 呼叫函式：這裡現在會印出「初始牌堆」與「洗牌後的牌堆」的所有 52 張牌
    initializeDeck(cardDeck);

    Player player, dealer;//宣告並初始化莊家以及玩家
    cout << "--- 遊戲開始 ---\n";
    initializePlayer(&player, "玩家", cardDeck);
    initializePlayer(&dealer, "莊家", cardDeck);

    playerTurn(&player, cardDeck);

    // 只有玩家沒爆牌，莊家才需要抽牌
    if (player.score <= 21) {
        cout << "\n--- 莊家回合 ---\n";
        dealerTurn(&dealer, cardDeck);
        cout << "\n=== 結算結果 ===\n";
        determineWinner(&player, &dealer);
    }
    return 0;
}
```

Discussion Section

1. Stack 堆疊：

Stack 遵循 **LIFO (Last-In, First-Out，後進先出)** 原則。

發牌邏輯：把洗好的牌一張一張疊起來時，最後放上去的那張牌，會在牌堆的最頂端。當要發牌時，也是從最頂端拿牌。

結果：最後放進牌堆的牌，最先被抽出來。

2. Queue 佇列：

Queue 遵循 **FIFO (First-In, First-Out，先進先出)** 原則。

發牌邏輯：把洗好的牌從後面一張一張塞進去，排成一列。發牌時，是從最下面把牌抽出來。

結果：最先放進牌堆的牌，最先被抽出來。。